

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I VENNA MEENAKSHI REDDY (192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Given relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

1. Understand the case

An online shopping platform stores order information. An order can contain multiple products, so a single OrderID cannot uniquely identify every row if each product in the order is stored separately.

Therefore, we assume:

- OrderID identifies an order.
- CustID identifies a customer.
- ProdID identifies a product.
- (OrderID, ProdID) identifies one particular product within an order.
- Qty is the quantity ordered.
- Price is the product's current/unit price.

Example

OrderID	CustID	CustName	ProdID	ProdName	Qty	Price
O101	C01	Ravi	P01	Keyboard	2	800
O101	C01	Ravi	P02	Mouse	1	500
O102	C02	Priya	P01	Keyboard	1	800

Here, O101 occurs twice because the order contains two products.

2. Functional Dependencies

The important functional dependencies are:

FD1

OrderID $\rightarrow$ CustID

An order belongs to one customer.

FD2

CustID $\rightarrow$ CustName

A customer ID determines the customer name.

FD3

$\text{ProdID} \rightarrow \text{ProdName}, \text{Price}$

A product ID determines its product name and price.

FD4

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

The quantity depends on the particular product in the particular order.

Therefore:

$\text{OrderID} \rightarrow \text{CustID}$

$\text{CustID} \rightarrow \text{CustName}$

$\text{ProdID} \rightarrow \text{ProdName}$

$\text{ProdID} \rightarrow \text{Price}$

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

Since:

$\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$

we also have the transitive dependency:

$\text{OrderID} \rightarrow \text{CustName}$

Types of Functional Dependencies**a. Full Functional Dependency**

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

Qty depends on both OrderID and ProdID. Neither attribute alone is sufficient to determine the quantity.

Therefore, this is a Full Functional Dependency.

b. Partial Functional Dependency

$\text{OrderID} \rightarrow \text{CustID}$

CustID depends only on OrderID, which is only part of the composite key (OrderID, ProdID).

Therefore, this is a Partial Dependency.

$\text{ProdID} \rightarrow \text{ProdName}$

$\text{ProdID} \rightarrow \text{Price}$

ProdName and Price depend only on ProdID, which is also only part of the composite key.

Therefore, these are Partial Dependencies.

c. Transitive Functional Dependency

$\text{OrderID} \rightarrow \text{CustID}$

$\text{CustID} \rightarrow \text{CustName}$

Therefore:

$\text{OrderID} \rightarrow \text{CustName}$

CustName depends on OrderID indirectly through CustID.

Therefore, this is a Transitive Dependency.

d. Trivial Functional Dependency

Examples:

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{OrderID}$

$(\text{OrderID}, \text{ProdID}) \rightarrow \text{ProdID}$

These are Trivial FDs because the attribute on the right side is already included in the left side.

e. Non-Trivial Functional Dependency

Examples:

$\text{OrderID} \rightarrow \text{CustID}$

$\text{ProdID} \rightarrow \text{ProdName}$

$\text{CustID} \rightarrow \text{CustName}$

These are Non-Trivial FDs because the right-side attribute is not part of the determinant.

3. Candidate Key

Because one order may contain several products:

Candidate Key = (OrderID, ProdID)

Why?

- OrderID alone cannot identify a row because an order can have multiple products.
- ProdID alone cannot identify a row because a product can occur in many orders.
- Together, (OrderID, ProdID) uniquely identify an order item.

4. Problems in the Original Table

The original relation has several anomalies.

Update anomaly

Suppose the product P01 is called "Keyboard".

If P01 occurs in 500 orders, the product name is stored 500 times.

If the name changes, all 500 records must be updated.

If one record is not updated, inconsistent data occurs.

Insertion anomaly

Suppose a new product P10 is added to the shopping platform but has not yet been ordered.

The original relation cannot easily store the product independently because an order is required.

Deletion anomaly

Suppose order O101 is the only order containing product P05.

If O101 is deleted, information about P05 may also be lost.

5. First Normal Form — 1NF

The relation is in 1NF if:

- Each attribute contains atomic values.
- There are no repeating groups.
- Each row represents one order-product combination.

The original table can therefore satisfy 1NF.

```
Order(  
  OrderID,  
  CustID,  
  CustName,  
  ProdID,  
  ProdName,  
  Qty,  
  Price  
)
```

Primary key:

(OrderID, ProdID)

6. Second Normal Form — 2NF

2NF requires:

The relation must be in 1NF and every non-key attribute must depend on the whole composite key.

But we have:

$\text{OrderID} \rightarrow \text{CustID}$

$\text{OrderID} \rightarrow \text{CustName}$

$\text{ProdID} \rightarrow \text{ProdName}$

$\text{ProdID} \rightarrow \text{Price}$

These attributes depend only on part of the composite key.

Therefore, partial dependencies exist.

Decomposition

Separate customer, product, and order-item information.

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

ORDERS(OrderID, CustID)

ORDER_ITEM(OrderID, ProdID, Qty)

7. Third Normal Form — 3NF

3NF requires:

No non-key attribute should depend transitively on a key.

In the original table:

$\text{OrderID} \rightarrow \text{CustID}$

$\text{CustID} \rightarrow \text{CustName}$

Therefore:

$\text{OrderID} \rightarrow \text{CustName}$

is a transitive dependency.

We remove it by creating:

CUSTOMER(CustID, CustName)

and:

ORDERS(OrderID, CustID)

Final 3NF Relations:

CUSTOMER

Customer(CustID, CustName)

- **PK:** CustID

PRODUCT

Product(ProdID, ProdName, Price)

- **PK:** ProdID

ORDERS

Orders(OrderID, CustID)

- **PK:** OrderID
- **FK:** CustID $\rightarrow$ Customer(CustID)

ORDER_ITEM

OrderItem(OrderID, ProdID, Qty)

- **Composite PK:** (OrderID, ProdID)
- **FK:** OrderID $\rightarrow$ Orders(OrderID)
- **FK:** ProdID $\rightarrow$ Product(ProdID)

Why this is 3NF

- Customer information depends only on CustID.
- Product information depends only on ProdID.
- Order information depends only on OrderID.
- Quantity depends on the complete key (OrderID, ProdID).
- No partial or transitive dependencies remain.

8. MySQL Implementation

```
MySQL 8.0 Command Line Cli  x  +  v

mysql> -- =====
mysql> -- Q1: ONLINE SHOPPING PLATFORM
mysql> -- NORMALIZATION UP TO 3NF
mysql> -- =====
mysql> DROP DATABASE IF EXISTS OnlineShoppingDB;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql>
mysql> CREATE DATABASE OnlineShoppingDB;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE OnlineShoppingDB;
Database changed
mysql>
mysql>
mysql> -- =====
mysql> -- STEP 1: CUSTOMER TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Customer (
  ->   CustID INT PRIMARY KEY,
  ->   CustName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 2: PRODUCT TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Product (
  ->   ProdID INT PRIMARY KEY,
  ->   ProdName VARCHAR(100) NOT NULL,
  ->   Price DECIMAL(10,2) NOT NULL,
  ->   CHECK (Price >= 0)
  -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql>
mysql> -- =====
mysql> -- STEP 3: ORDERS TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Orders (
  ->   OrderID INT PRIMARY KEY,
  ->   CustID INT NOT NULL,
  ->
  ->   CONSTRAINT fk_order_customer
  ->   FOREIGN KEY (CustID)
  ->   REFERENCES Customer(CustID)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 4: ORDER ITEM TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE OrderItem (
  ->   OrderID INT,
  ->   ProdID INT,
  ->   Qty INT NOT NULL,
  ->
  ->   PRIMARY KEY (OrderID, ProdID),
  ->
  ->   CONSTRAINT fk_item_order
  ->   FOREIGN KEY (OrderID)
  ->   REFERENCES Orders(OrderID),
  ->
  ->   CONSTRAINT fk_item_product
  ->   FOREIGN KEY (ProdID)
  ->   REFERENCES Product(ProdID),
  ->
  ->   CHECK (Qty > 0)
  -> );
Query OK, 0 rows affected (0.06 sec)
```

```
MySQL 8.0 Command Line Cli
mysql>
mysql> -- =====
mysql> -- INSERT SAMPLE CUSTOMERS
mysql> -- =====
mysql>
mysql> INSERT INTO Customer VALUES
-> (1, 'Ravi'),
-> (2, 'Priya'),
-> (3, 'Arun');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- INSERT SAMPLE PRODUCTS
mysql> -- =====
mysql>
mysql> INSERT INTO Product VALUES
-> (101, 'Keyboard', 800.00),
-> (102, 'Mouse', 500.00),
-> (103, 'Monitor', 12000.00),
-> (104, 'Headphones', 1500.00);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- INSERT ORDERS
mysql> -- =====
mysql>
mysql> INSERT INTO Orders VALUES
-> (1001, 1),
-> (1002, 2),
-> (1003, 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- INSERT ORDER ITEMS
mysql> -- =====
mysql>
mysql> INSERT INTO OrderItem VALUES
-> (1001, 101, 2),
-> (1001, 102, 1),
-> (1002, 103, 1),
-> (1003, 104, 2);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli
mysql> -- =====
mysql> -- DISPLAY TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Customer;
+----+-----+
| CustID | CustName |
+----+-----+
| 1      | Ravi     |
| 2      | Priya    |
| 3      | Arun     |
+----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| 101    | Keyboard | 800.00 |
| 102    | Mouse   | 500.00 |
| 103    | Monitor  | 12000.00 |
| 104    | Headphones | 1500.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Orders;
+-----+-----+
| OrderID | CustID |
+-----+-----+
| 1001    | 1      |
| 1003    | 1      |
| 1002    | 2      |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM OrderItem;
+-----+-----+-----+
| OrderID | ProdID | Qty |
+-----+-----+-----+
| 1001    | 101    | 2    |
| 1001    | 102    | 1    |
| 1002    | 103    | 1    |
| 1003    | 104    | 2    |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli × + v

mysql> -- =====
mysql> -- DISPLAY COMPLETE ORDER DETAILS
mysql> -- =====
mysql>
mysql> SELECT
-> o.OrderID,
-> c.CustID,
-> c.CustName,
-> p.ProdID,
-> p.ProdName,
-> oi.Qty,
-> p.Price
-> FROM Orders o
-> JOIN Customer c
-> ON o.CustID = c.CustID
-> JOIN OrderItem oi
-> ON o.OrderID = oi.OrderID
-> JOIN Product p
-> ON oi.ProdID = p.ProdID
-> ORDER BY o.OrderID;

+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+-----+
| 1001 | 1 | Ravi | 101 | Keyboard | 2 | 800.00 |
| 1001 | 1 | Ravi | 102 | Mouse | 1 | 500.00 |
| 1002 | 2 | Priya | 103 | Monitor | 1 | 12000.00 |
| 1003 | 1 | Ravi | 104 | Headphones | 2 | 1500.00 |
+-----+-----+-----+-----+-----+-----+

4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- FIND TOTAL VALUE OF EACH ORDER
mysql> -- =====
mysql>
mysql> SELECT
-> o.OrderID,
-> c.CustName,
-> SUM(oi.Qty * p.Price) AS TotalAmount
-> FROM Orders o
-> JOIN Customer c
-> ON o.CustID = c.CustID
-> JOIN OrderItem oi
-> ON o.OrderID = oi.OrderID
-> JOIN Product p
-> ON oi.ProdID = p.ProdID
-> GROUP BY o.OrderID, c.CustName;

+-----+-----+-----+
| OrderID | CustName | TotalAmount |
+-----+-----+-----+
| 1001 | Ravi | 2100.00 |
| 1003 | Ravi | 3000.00 |
| 1002 | Priya | 12000.00 |
+-----+-----+-----+

3 rows in set (0.00 sec)

mysql> |
```

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Given relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

1. Understand the Case

Assume:

- Each patient has one patient ID.
- Each patient is assigned to one doctor.
- Each doctor has one doctor ID.
- Each doctor has one name and one specialization.
- Each patient is assigned to one ward.
- Each ward has one ward number and one ward name.

Example:

PID	PName	DocID	DocName	Specialization	WardNo
1	Ravi	D01	Dr. Kumar	Cardiology	W01
2	Priya	D02	Dr. Meena	Neurology	W02
3	Arun	D01	Dr. Kumar	Cardiology	W01

Notice that doctor and ward information is repeatedly stored.

2. Functional Dependencies

The major functional dependencies are:

$PID \rightarrow PName, DocID, WardNo$

$DocID \rightarrow DocName, Specialization$

$WardNo \rightarrow WardName$

We can also derive:

$PID \rightarrow DocID \rightarrow DocName$

$PID \rightarrow DocID \rightarrow Specialization$

$PID \rightarrow WardNo \rightarrow WardName$

Therefore, there are transitive dependencies.

Types of Functional Dependencies

1. Full Functional Dependency

$(\text{SID}, \text{CourseID}) \rightarrow \text{Grade}$

Grade depends on both the student and the course, so it is a Full Functional Dependency.

2. Partial Functional Dependency

$\text{SID} \rightarrow \text{SName}$

$\text{CourseID} \rightarrow \text{CourseName}$

$\text{CourseID} \rightarrow \text{Faculty}$

These are Partial Functional Dependencies because the attributes depend on only part of the composite key (SID, CourseID).

3. Transitive Functional Dependency

No transitive functional dependency is explicitly present in the given relation.

The important dependencies are directly:

$\text{SID} \rightarrow \text{SName}$

$\text{CourseID} \rightarrow \text{CourseName}, \text{Faculty}$

So the main problem is partial dependency, which is why the relation must be decomposed to 2NF.

4. Trivial Functional Dependency

$(\text{SID}, \text{CourseID}) \rightarrow \text{SID}$

$(\text{SID}, \text{CourseID}) \rightarrow \text{CourseID}$

These are Trivial Functional Dependencies because the right-hand attribute is already included in the left-hand side.

5. Non-Trivial Functional Dependency

$\text{SID} \rightarrow \text{SName}$

$\text{CourseID} \rightarrow \text{CourseName}$

$\text{CourseID} \rightarrow \text{Faculty}$

These are Non-Trivial Functional Dependencies because the right-hand-side attributes are not part of the determinant.

3. Candidate Key

The candidate key is:

PID

because:

$\text{PID} \rightarrow \text{PName}, \text{DocID}, \text{WardNo}$

$\text{DocID} \rightarrow \text{DocName}, \text{Specialization}$

$\text{WardNo} \rightarrow \text{WardName}$

Therefore, PID determines all attributes.

4. Anomalies

Update anomaly

Suppose Dr. Kumar's specialization changes.

The original table may contain Dr. Kumar in hundreds of patient records.

Every occurrence must be updated.

If some records remain unchanged, inconsistent information results.

Insertion anomaly

Suppose a new doctor joins the hospital but has no patient yet.

The original relation cannot store the doctor independently because PID is required.

Similarly, a new ward cannot easily be inserted without a patient.

Deletion anomaly

Suppose the only patient assigned to Ward W05 is discharged.

Deleting that patient could remove the only stored information about Ward W05.

Similarly, if a doctor has only one patient and that patient is deleted, doctor information could be lost.

5. BCNF Analysis

BCNF states:

For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.

Consider:

$\text{DocID} \rightarrow \text{DocName, Specialization}$

DocID is not a key of the original Patient relation because it does not determine the patient.

Therefore, this violates BCNF.

Similarly:

$\text{WardNo} \rightarrow \text{WardName}$

WardNo is not a superkey of the original relation.

Therefore, this also violates BCNF.

6. BCNF Decomposition

Separate the independent entities.

Patient

PATIENT(PID, PName, DocID, WardNo)

Doctor

DOCTOR(DocID, DocName, Specialization)

Ward

WARD(WardNo, WardName)

7. Verify BCNF

Doctor

DocID $\rightarrow$ DocName, Specialization

DocID is the primary key.

Therefore, the determinant is a superkey.

So Doctor is in BCNF.

Ward

WardNo $\rightarrow$ WardName

WardNo is the primary key.

Therefore, Ward is in BCNF.

Patient

PID $\rightarrow$ PName, DocID, WardNo

PID is the primary key.

Therefore, Patient is in BCNF.

Final BCNF schema

BCNF Decomposition

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

PATIENT(PID, PName, DocID, WardNo)

- DocID is the Primary Key (PK) of DOCTOR.
- WardNo is the Primary Key (PK) of WARD.
- PID is the Primary Key (PK) of PATIENT.
- DocID in PATIENT is a Foreign Key (FK) referencing DOCTOR(DocID).
- WardNo in PATIENT is a Foreign Key (FK) referencing WARD(WardNo).
- The functional dependencies $\text{DocID} \rightarrow \text{DocName, Specialization}$ and $\text{WardNo} \rightarrow \text{WardName}$ are satisfied because DocID and WardNo are keys in their respective relations.
- The decomposition removes update, insertion, and deletion anomalies and eliminates the BCNF violations present in the original relation.
- Therefore, the resulting relations satisfy BCNF.

8. MySQL Implementation

```
MySQL 8.0 Command Line Cli  X  +  v

mysql>
mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE HospitalDB;
Database changed
mysql>
mysql>
mysql> -- =====
mysql> -- STEP 1: DOCTOR TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Doctor (
  ->     DocID INT PRIMARY KEY,
  ->     DocName VARCHAR(100) NOT NULL,
  ->     Specialization VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 2: WARD TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Ward (
  ->     WardNo INT PRIMARY KEY,
  ->     WardName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 3: PATIENT TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Patient (
  ->     PID INT PRIMARY KEY,
  ->     PName VARCHAR(100) NOT NULL,
  ->     DocID INT NOT NULL,
  ->     WardNo INT NOT NULL,
  ->
  ->     CONSTRAINT fk_patient_doctor
  ->         FOREIGN KEY (DocID)
  ->         REFERENCES Doctor(DocID),
  ->
  ->     CONSTRAINT fk_patient_ward
  ->         FOREIGN KEY (WardNo)
  ->         REFERENCES Ward(WardNo)
  -> );
Query OK, 0 rows affected (0.07 sec)
```



```
MySQL 8.0 Command Line Cli × + v

mysql> -- INSERT DOCTORS
mysql> -- =====
mysql> INSERT INTO Doctor VALUES
--> (101, 'Dr. Kumar', 'Cardiology'),
--> (102, 'Dr. Meena', 'Neurology'),
--> (103, 'Dr. Raj', 'Orthopedics');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- INSERT WARDS
mysql> -- =====
mysql> INSERT INTO Ward VALUES
--> (1, 'Cardiac Ward'),
--> (2, 'Neurology Ward'),
--> (3, 'Orthopedic Ward');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- INSERT PATIENTS
mysql> -- =====
mysql> INSERT INTO Patient VALUES
--> (1001, 'Ravi', 101, 1),
--> (1002, 'Priya', 102, 2),
--> (1003, 'Arun', 101, 1),
--> (1004, 'Kiran', 103, 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql>
mysql> -- =====
mysql> -- DISPLAY TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| 101   | Dr. Kumar | Cardiology     |
| 102   | Dr. Meena | Neurology      |
| 103   | Dr. Raj   | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Ward;
+-----+-----+
| WardNo | WardName |
+-----+-----+
| 1       | Cardiac Ward |
| 2       | Neurology Ward |
| 3       | Orthopedic Ward |
+-----+-----+
3 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli × + v

mysql> SELECT * FROM Patient;
+----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+----+-----+-----+-----+
| 1001 | Ravi  | 101   | 1       |
| 1002 | Priya | 102   | 2       |
| 1003 | Arun  | 101   | 1       |
| 1004 | Kiran | 103   | 3       |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE PATIENT INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
--> p.PID,
--> p.PName,
--> d.DocID,
--> d.DocName,
--> d.Specialization,
--> w.WardNo,
--> w.WardName
--> FROM Patient p
--> JOIN Doctor d
--> ON p.DocID = d.DocID
--> JOIN Ward w
--> ON p.WardNo = w.WardNo
--> ORDER BY p.PID;
+----+-----+-----+-----+-----+-----+
| PID | PName | DocID | DocName | Specialization | WardNo | WardName |
+----+-----+-----+-----+-----+-----+
| 1001 | Ravi  | 101   | Dr. Kumar | Cardiology     | 1       | Cardiac Ward |
| 1002 | Priya | 102   | Dr. Meena | Neurology      | 2       | Neurology Ward |
| 1003 | Arun  | 101   | Dr. Kumar | Cardiology     | 1       | Cardiac Ward |
| 1004 | Kiran | 103   | Dr. Raj   | Orthopedics    | 3       | Orthopedic Ward |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- FIND PATIENTS UNDER EACH DOCTOR
mysql> -- =====
mysql>
mysql> SELECT
--> d.DocID,
--> d.DocName,
--> COUNT(p.PID) AS NumberOfPatients
--> FROM Doctor d
--> LEFT JOIN Patient p
--> ON d.DocID = p.DocID
--> GROUP BY d.DocID, d.DocName;
+-----+-----+-----+
| DocID | DocName | NumberOfPatients |
+-----+-----+-----+
| 101   | Dr. Kumar | 2                |
| 102   | Dr. Meena | 1                |
| 103   | Dr. Raj   | 1                |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql>
mysql>
mysql> -- =====
mysql> -- FIND NUMBER OF PATIENTS IN EACH WARD
mysql> -- =====
mysql>
mysql> SELECT
mysql>   -> w.WardNo,
mysql>   -> w.WardName,
mysql>   -> COUNT(p.PID) AS NumberOfPatients
mysql>   -> FROM Ward w
mysql>   -> LEFT JOIN Patient p
mysql>   -> ON w.WardNo = p.WardNo
mysql>   -> GROUP BY w.WardNo, w.WardName;
```

WardNo	WardName	NumberOfPatients
1	Cardiac Ward	2
2	Neurology Ward	1
3	Orthopedic Ward	1

3 rows in set (0.00 sec)

```
mysql> |
```

3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Given relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

1. Understand the Case

A university stores student enrollment information.

Assume:

- SID identifies a student.
- CourseID identifies a course.
- A student can enroll in many courses.
- A course can contain many students.
- Grade represents the grade obtained by a student in a course.

Example:

SID	SName	CourseID	CourseName	Faculty	Grade
S01	Ravi	C01	DBMS	Dr. Kumar	A
S01	Ravi	C02	Java	Dr. Meena	B+
S02	Priya	C01	DBMS	Dr. Kumar	A+

2. Functional Dependencies

The functional dependencies are:

Student dependency

$SID \rightarrow SName$

A student ID determines the student name.

Course dependency

$CourseID \rightarrow CourseName, Faculty$

A course ID determines the course name and faculty.

Enrollment dependency

$(SID, CourseID) \rightarrow Grade$

The grade depends on both the student and course.

Therefore:

$SID \rightarrow SName$

$\text{CourseID} \rightarrow \text{CourseName, Faculty}$

$(\text{SID, CourseID}) \rightarrow \text{Grade}$

Types of Functional Dependencies

Given:

$\text{Enrollment}(\text{SID, SName, CourseID, CourseName, Faculty, Grade})$

1. Full Functional Dependency

$(\text{SID, CourseID}) \rightarrow \text{Grade}$

2. Partial Functional Dependency

$\text{SID} \rightarrow \text{SName}$

$\text{CourseID} \rightarrow \text{CourseName}$

$\text{CourseID} \rightarrow \text{Faculty}$

3. Transitive Functional Dependency

No transitive dependency is explicitly present in the given relation.

4. Trivial Functional Dependency

$(\text{SID, CourseID}) \rightarrow \text{SID}$

$(\text{SID, CourseID}) \rightarrow \text{CourseID}$

5. Non-Trivial Functional Dependency

$\text{SID} \rightarrow \text{SName}$

$\text{CourseID} \rightarrow \text{CourseName}$

$\text{CourseID} \rightarrow \text{Faculty}$

3. Candidate Key

The candidate key is:

(SID, CourseID)

Why?

A student can take multiple courses, and a course can have multiple students.

Therefore:

$\text{SID alone} \rightarrow \text{cannot uniquely identify a record}$

$\text{CourseID alone} \rightarrow \text{cannot uniquely identify a record}$

$\text{SID + CourseID} \rightarrow \text{uniquely identifies enrollment}$

4. 1NF

The table is in 1NF when:

- Values are atomic.
- Each row represents one student-course enrollment.
- There are no repeating groups.

So:

```
Enrollment(  
    SID,  
    SName,  
    CourseID,  
    CourseName,  
    Faculty,  
    Grade  
)
```

with:

Primary Key = (SID, CourseID)

5. Identify Partial Dependencies

Because the primary key is composite:

(SID, CourseID)

we must check whether non-key attributes depend on the entire key.

But:

$SID \rightarrow SName$

SName depends only on SID.

This is a partial dependency.

Also:

$CourseID \rightarrow CourseName, Faculty$

These attributes depend only on CourseID.

These are also partial dependencies.

Therefore, the original table is not in 2NF.

6. Decompose to 2NF

Separate student information:

STUDENT(SID, SName)

Separate course information:

COURSE(CourseID, CourseName, Faculty)

Keep enrollment-specific information:

ENROLLMENT(SID, CourseID, Grade)

7. Verify 2NF

Student

$SID \rightarrow SName$

SID is the complete primary key.

No partial dependency exists.

Course

$CourseID \rightarrow CourseName, Faculty$

CourseID is the complete primary key.

No partial dependency exists.

Enrollment

$(SID, CourseID) \rightarrow Grade$

Grade depends on the complete composite key.

Therefore, the decomposition satisfies 2NF.

8. Final 2NF Schema

STUDENT

STUDENT(SID, SName)

- **SID** – Primary Key (PK)
- **SName** – Student name

COURSE

COURSE(CourseID, CourseName, Faculty)

- **CourseID** – Primary Key (PK)
- **CourseName** – Course name
- **Faculty** – Faculty/Instructor

ENROLLMENT

ENROLLMENT(SID, CourseID, Grade)

- **SID** – Primary Key (PK), Foreign Key (FK) $\rightarrow$ STUDENT(SID)
- **CourseID** – Primary Key (PK), Foreign Key (FK) $\rightarrow$ COURSE(CourseID)

- **Grade** – Grade obtained by the student

Composite Primary Key: (SID, CourseID)

Relationship

STUDENT 1 —————< ENROLLMENT >————— 1 COURSE

The ENROLLMENT table resolves the many-to-many relationship between students and courses.

A student can enroll in many courses, and a course can have many students.

Conclusion

The original relation contained partial dependencies:

SID → SName

CourseID → CourseName, Faculty

These dependencies are removed by decomposing the relation into STUDENT, COURSE, and ENROLLMENT. The remaining Grade attribute depends on the complete composite key (SID, CourseID). Therefore, the resulting relations are in 2NF.

9. MySQL Implementation

```
mysql>
mysql> USE UniversityDB;
Database changed
mysql>
mysql>
mysql> -- =====
mysql> -- STEP 1: STUDENT TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Student (
->   SID INT PRIMARY KEY,
->   SName VARCHAR(100) NOT NULL
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 2: COURSE TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Course (
->   CourseID INT PRIMARY KEY,
->   CourseName VARCHAR(100) NOT NULL,
->   Faculty VARCHAR(100) NOT NULL
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- STEP 3: ENROLLMENT TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Enrollment (
->   SID INT,
->   CourseID INT,
->   Grade VARCHAR(5),
->   PRIMARY KEY (SID, CourseID),
->   CONSTRAINT fk_enrollment_student
->     FOREIGN KEY (SID)
->     REFERENCES Student(SID),
->   CONSTRAINT fk_enrollment_course
->     FOREIGN KEY (CourseID)
->     REFERENCES Course(CourseID)
-> );
Query OK, 0 rows affected (0.06 sec)
```

```
MySQL 8.0 Command Line Cli × + ▾

mysql> -- =====
mysql> -- INSERT STUDENTS
mysql> -- =====
mysql>
mysql> INSERT INTO Student VALUES
    → (1, 'Ravi'),
    → (2, 'Priya'),
    → (3, 'Arun'),
    → (4, 'Kiran');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT COURSES
mysql> -- =====
mysql>
mysql> INSERT INTO Course VALUES
    → (101, 'Database Management Systems', 'Dr. Kumar'),
    → (102, 'Java Programming', 'Dr. Meena'),
    → (103, 'Computer Networks', 'Dr. Raj');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT ENROLLMENTS
mysql> -- =====
mysql>
mysql> INSERT INTO Enrollment VALUES
    → (1, 101, 'A'),
    → (1, 102, 'B+'),
    → (2, 101, 'A+'),
    → (2, 103, 'A'),
    → (3, 102, 'B'),
    → (4, 103, 'A');
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- DISPLAY TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Student;
+----+-----+
| SID | SName |
+----+-----+
| 1   | Ravi  |
| 2   | Priya |
| 3   | Arun  |
| 4   | Kiran |
+----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Course;
+----+-----+-----+
| CourseID | CourseName | Faculty |
+----+-----+-----+
| 101      | Database Management Systems | Dr. Kumar |
| 102      | Java Programming | Dr. Meena |
| 103      | Computer Networks | Dr. Raj |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Enrollment;
+----+-----+-----+
| SID | CourseID | Grade |
+----+-----+-----+
| 1   | 101      | A      |
| 1   | 102      | B+     |
| 2   | 101      | A+     |
| 2   | 103      | A      |
| 3   | 102      | B      |
| 4   | 103      | A      |
+----+-----+-----+
6 rows in set (0.00 sec)
```



```
MySQL 8.0 Command Line Cli  ×  +  ∨

mysql> -- =====
mysql> -- DISPLAY COMPLETE ENROLLMENT INFORMATION
mysql> -- =====
mysql> SELECT
-> s.SID,
-> s.SName,
-> c.CourseID,
-> c.CourseName,
-> c.Faculty,
-> e.Grade
-> FROM Enrollment e
-> JOIN Student s
-> ON e.SID = s.SID
-> JOIN Course c
-> ON e.CourseID = c.CourseID
-> ORDER BY s.SID, c.CourseID;

+----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+----+-----+-----+-----+-----+
| 1 | Ravi | 101 | Database Management Systems | Dr. Kumar | A |
| 1 | Ravi | 102 | Java Programming | Dr. Meena | B+ |
| 2 | Priya | 101 | Database Management Systems | Dr. Kumar | A+ |
| 2 | Priya | 103 | Computer Networks | Dr. Raj | A |
| 3 | Arun | 102 | Java Programming | Dr. Meena | B |
| 4 | Kiran | 103 | Computer Networks | Dr. Raj | A |
+----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- FIND COURSES TAKEN BY EACH STUDENT
mysql> -- =====
mysql> SELECT
-> s.SID,
-> s.SName,
-> c.CourseName,
-> e.Grade
-> FROM Student s
-> JOIN Enrollment e
-> ON s.SID = e.SID
-> JOIN Course c
-> ON e.CourseID = c.CourseID
-> ORDER BY s.SID;

+----+-----+-----+-----+
| SID | SName | CourseName | Grade |
+----+-----+-----+-----+
| 1 | Ravi | Database Management Systems | A |
| 1 | Ravi | Java Programming | B+ |
| 2 | Priya | Database Management Systems | A+ |
| 2 | Priya | Computer Networks | A |
| 3 | Arun | Java Programming | B |
| 4 | Kiran | Computer Networks | A |
+----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- FIND NUMBER OF STUDENTS IN EACH COURSE
mysql> -- =====
mysql> SELECT
-> c.CourseID,
-> c.CourseName,
-> COUNT(e.SID) AS NumberOfStudents
-> FROM Course c
-> LEFT JOIN Enrollment e
-> ON c.CourseID = e.CourseID
-> GROUP BY c.CourseID, c.CourseName;

+----+-----+-----+
| CourseID | CourseName | NumberOfStudents |
+----+-----+-----+
| 101 | Database Management Systems | 2 |
| 102 | Java Programming | 2 |
| 103 | Computer Networks | 2 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> -- =====
mysql> -- FIND STUDENTS WHO GOT GRADE A
mysql> -- =====
mysql>
mysql> SELECT
-> s.SID,
-> s.SName,
-> c.CourseName,
-> e.Grade
-> FROM Enrollment e
-> JOIN Student s
-> ON e.SID = s.SID
-> JOIN Course c
-> ON e.CourseID = c.CourseID
-> WHERE e.Grade IN ('A', 'A+');

+----+-----+-----+-----+
| SID | SName | CourseName | Grade |
+----+-----+-----+-----+
| 1 | Ravi | Database Management Systems | A |
| 2 | Priya | Database Management Systems | A+ |
| 2 | Priya | Computer Networks | A |
| 4 | Kiran | Computer Networks | A |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Given relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

1. Case Analysis

An airline booking system stores information about flights and passengers.

Assume:

- FlightNo uniquely identifies a flight.
- A flight has one departure city, arrival city, and scheduled date.
- PassID uniquely identifies a passenger.
- A passenger has one passenger name.
- A passenger can book multiple flights.
- A seat number is assigned to a passenger for a particular flight.
- (FlightNo, PassID) identifies one booking.

Example:

FlightNo	PassID	PassName	Seat	DeptCity	ArrCity	Date
F101	P01	Ravi	12A	Chennai	Delhi	2026-08-15
F102	P02	Priya	15B	Mumbai	Delhi	2026-08-16
F101	P03	Arun	12B	Chennai	Delhi	2026-08-15

2. Functional Dependencies

The important FDs are:

FlightNo $\rightarrow$ DeptCity, ArrCity, Date

PassID $\rightarrow$ PassName

(FlightNo, PassID) $\rightarrow$ Seat

Therefore:

FlightNo $\rightarrow$ DeptCity

FlightNo $\rightarrow$ ArrCity

FlightNo $\rightarrow$ Date

PassID $\rightarrow$ PassName

(FlightNo, PassID) $\rightarrow$ Seat

3. Candidate Key

The candidate key is:

(FlightNo, PassID)

because one flight can have many passengers and one passenger can book many flights.

Neither FlightNo nor PassID alone identifies a booking.

4. Anomalies

Update anomaly

If the departure city of flight F101 changes, every booking for F101 must be updated.

Failure to update all rows produces inconsistent flight information.

Insertion anomaly

A new flight cannot easily be stored if there are no bookings because the relation is centered around bookings.

Deletion anomaly

If the last passenger booking for a flight is deleted, information about the flight itself may also be lost.

5. BCNF Analysis

BCNF requires:

For every non-trivial FD $X \rightarrow Y$, X must be a superkey.

Consider:

$\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}, \text{Date}$

FlightNo is not a superkey of the original Booking relation because a flight can have many passengers.

Therefore, this violates BCNF.

Also:

$\text{PassID} \rightarrow \text{PassName}$

PassID is not a superkey of the original relation.

Therefore, this also violates BCNF.

6. BCNF Decomposition

Create separate relations:

FLIGHT(FlightNo, DeptCity, ArrCity, Date)

PASSENGER(PassID, PassName)

BOOKING(FlightNo, PassID, Seat)

Final BCNF Schema

FLIGHT

FLIGHT(FlightNo, DeptCity, ArrCity, Date)

- FlightNo – PK

PASSENGER

PASSENGER(PassID, PassName)

- PassID – PK

BOOKING

BOOKING(FlightNo, PassID, Seat)

- Composite PK: (FlightNo, PassID)
- FlightNo – FK $\rightarrow$ FLIGHT(FlightNo)
- PassID – FK $\rightarrow$ PASSENGER(PassID)

Why it is BCNF

In FLIGHT:

FlightNo $\rightarrow$ DeptCity, ArrCity, Date

FlightNo is the primary key.

In PASSENGER:

PassID $\rightarrow$ PassName

PassID is the primary key.

In BOOKING:

(FlightNo, PassID) $\rightarrow$ Seat

The determinant is the composite primary key.

Therefore, all determinants are superkeys, so the decomposed relations satisfy BCNF.

7. MySQL Implementation

```
mysql>
mysql> CREATE TABLE Flight (
->     FlightNo VARCHAR(10) PRIMARY KEY,
->     DeptCity VARCHAR(100) NOT NULL,
->     ArrCity VARCHAR(100) NOT NULL,
->     FlightDate DATE NOT NULL
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- =====
mysql> -- STEP 2: PASSENGER TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Passenger (
->     PassID VARCHAR(10) PRIMARY KEY,
->     PassName VARCHAR(100) NOT NULL
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- =====
mysql> -- STEP 3: BOOKING TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Booking (
->     FlightNo VARCHAR(10),
->     PassID VARCHAR(10),
->     Seat VARCHAR(10) NOT NULL,
->
->     PRIMARY KEY (FlightNo, PassID),
->
->     CONSTRAINT fk_booking_flight
->         FOREIGN KEY (FlightNo)
->         REFERENCES Flight(FlightNo),
->
->     CONSTRAINT fk_booking_passenger
->         FOREIGN KEY (PassID)
->         REFERENCES Passenger(PassID)
-> );
Query OK, 0 rows affected (0.07 sec)
```

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> -- =====
mysql> -- INSERT FLIGHT DATA
mysql> -- =====
mysql>
mysql> INSERT INTO Flight VALUES
-> ('F101', 'Chennai', 'Delhi', '2026-08-15'),
-> ('F102', 'Mumbai', 'Delhi', '2026-08-16'),
-> ('F103', 'Bangalore', 'Kolkata', '2026-08-17');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT PASSENGER DATA
mysql> -- =====
mysql>
mysql> INSERT INTO Passenger VALUES
-> ('P01', 'Ravi'),
-> ('P02', 'Priya'),
-> ('P03', 'Arun'),
-> ('P04', 'Kiran');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT BOOKING DATA
mysql> -- =====
mysql>
mysql> INSERT INTO Booking VALUES
-> ('F101', 'P01', '12A'),
-> ('F101', 'P03', '12B'),
-> ('F102', 'P02', '15A'),
-> ('F103', 'P04', '10C');
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli x + v
mysql> -- =====
mysql> -- DISPLAY FINAL BCNF TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Flight;
+-----+-----+-----+-----+
| FlightNo | DeptCity | ArrCity | FlightDate |
+-----+-----+-----+-----+
| F101     | Chennai  | Delhi   | 2026-08-15 |
| F102     | Mumbai   | Delhi   | 2026-08-16 |
| F103     | Bangalore | Kolkata  | 2026-08-17 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Passenger;
+-----+-----+
| PassID | PassName |
+-----+-----+
| P01    | Ravi     |
| P02    | Priya    |
| P03    | Arun     |
| P04    | Kiran    |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Booking;
+-----+-----+-----+
| FlightNo | PassID | Seat |
+-----+-----+-----+
| F101     | P01    | 12A  |
| F101     | P03    | 12B  |
| F102     | P02    | 15A  |
| F103     | P04    | 10C  |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE BOOKING INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
  ->     b.FlightNo,
  ->     p.PassID,
  ->     p.PassName,
  ->     b.Seat,
  ->     f.DeptCity,
  ->     f.ArrCity,
  ->     f.FlightDate
  -> FROM Booking b
  -> JOIN Passenger p
  ->     ON b.PassID = p.PassID
  -> JOIN Flight f
  ->     ON b.FlightNo = f.FlightNo
  -> ORDER BY b.FlightNo, b.PassID;
+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | FlightDate |
+-----+-----+-----+-----+-----+-----+
| F101     | P01    | Ravi     | 12A  | Chennai  | Delhi   | 2026-08-15 |
| F101     | P03    | Arun     | 12B  | Chennai  | Delhi   | 2026-08-15 |
| F102     | P02    | Priya    | 15A  | Mumbai   | Delhi   | 2026-08-16 |
| F103     | P04    | Kiran    | 10C  | Bangalore | Kolkata  | 2026-08-17 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Assumed HR Relation

Since the question does not specify the attributes, assume the multinational HR system stores:

```
EMPLOYEE(  
  EmpID,  
  EmpName,  
  DeptID,  
  DeptName,  
  ManagerID,  
  ManagerName,  
  CountryID,  
  CountryName,  
  JobID,  
  JobTitle,  
  Salary  
)
```

This is a reasonable HR schema for analyzing functional and transitive dependencies.

1. Functional Dependencies

Employee dependency

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}, \text{CountryID}, \text{JobID}, \text{Salary}$

An employee ID determines the employee's information.

Department dependency

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

A department ID determines the department name and its manager.

Manager dependency

$\text{ManagerID} \rightarrow \text{ManagerName}$

A manager ID determines the manager name.

Country dependency

$\text{CountryID} \rightarrow \text{CountryName}$

A country ID determines the country name.

Job dependency

$\text{JobID} \rightarrow \text{JobTitle}$

A job ID determines the job title.

2. Transitive Dependencies

From:

$\text{EmpID} \rightarrow \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}$

we obtain:

$\text{EmpID} \rightarrow \text{DeptName}$

This is a transitive dependency.

Similarly:

$\text{EmpID} \rightarrow \text{ManagerID}$

$\text{ManagerID} \rightarrow \text{ManagerName}$

Therefore:

$\text{EmpID} \rightarrow \text{ManagerName}$

is transitive.

Also:

$\text{EmpID} \rightarrow \text{CountryID}$

$\text{CountryID} \rightarrow \text{CountryName}$

Therefore:

$\text{EmpID} \rightarrow \text{CountryName}$

is transitive.

And:

$\text{EmpID} \rightarrow \text{JobID}$

$\text{JobID} \rightarrow \text{JobTitle}$

Therefore:

$\text{EmpID} \rightarrow \text{JobTitle}$

is transitive.

3. Candidate Key

Assuming every employee has a unique ID:

Candidate Key = EmpID

4. Problems in the Original Relation

Update anomaly

If a department name changes, every employee belonging to that department may need to be updated.

Insertion anomaly

A new department cannot easily be stored if it has no employees.

Deletion anomaly

Deleting the last employee in a department could remove the only stored information about that department.

5. 3NF Decomposition

Separate employee information:

```
EMPLOYEE(  
    EmpID,  
    EmpName,  
    DeptID,  
    ManagerID,  
    CountryID,  
    JobID,  
    Salary  
)
```

Department information:

```
DEPARTMENT(  
    DeptID,  
    DeptName,  
    ManagerID  
)
```

Manager information:

```
MANAGER(  
    ManagerID,  
    ManagerName  
)
```

Country information:

```
COUNTRY(  
    CountryID,  
    CountryName  
)
```

Job information:

```
JOB(  
    JobID,  
    JobTitle  
)
```

Final 3NF Schema

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, CountryID, JobID, Salary)

- EmpID – PK
- DeptID – FK
- ManagerID – FK
- CountryID – FK
- JobID – FK

DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

- DeptID – PK
- ManagerID – FK

MANAGER

MANAGER(ManagerID, ManagerName)

- ManagerID – PK

COUNTRY

COUNTRY(CountryID, CountryName)

- CountryID – PK

JOB

JOB(JobID, JobTitle)

- JobID – PK

Why the decomposition is in 3NF

Each non-key attribute depends on the key of its own relation.

The transitive dependencies have been removed:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

$\text{EmpID} \rightarrow \text{CountryID} \rightarrow \text{CountryName}$

$\text{EmpID} \rightarrow \text{JobID} \rightarrow \text{JobTitle}$

Therefore, the final relations satisfy 3NF.

6. MySQL Implementation

```
mysql>
mysql> =====
mysql> -- STEP 1: MANAGER TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Manager (
  ->   ManagerID INT PRIMARY KEY,
  ->   ManagerName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> =====
mysql> -- STEP 2: COUNTRY TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Country (
  ->   CountryID INT PRIMARY KEY,
  ->   CountryName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> =====
mysql> -- STEP 3: JOB TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Job (
  ->   JobID INT PRIMARY KEY,
  ->   JobTitle VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> =====
mysql> -- STEP 4: DEPARTMENT TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Department (
  ->   DeptID INT PRIMARY KEY,
  ->   DeptName VARCHAR(100) NOT NULL,
  ->   ManagerID INT,
  ->   CONSTRAINT fk_department_manager
  ->     FOREIGN KEY (ManagerID)
  ->     REFERENCES Manager(ManagerID)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> =====
mysql> -- STEP 5: EMPLOYEE TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Employee (
  ->   EmpID INT PRIMARY KEY,
  ->   EmpName VARCHAR(100) NOT NULL,
  ->   DeptID INT,
  ->   ManagerID INT,
  ->   CountryID INT,
  ->   JobID INT,
  ->   Salary DECIMAL(10,2),
  ->   CONSTRAINT fk_employee_department
  ->     FOREIGN KEY (DeptID)
  ->     REFERENCES Department(DeptID),
  ->   CONSTRAINT fk_employee_manager
  ->     FOREIGN KEY (ManagerID)
  ->     REFERENCES Manager(ManagerID),
  ->   CONSTRAINT fk_employee_country
  ->     FOREIGN KEY (CountryID)
  ->     REFERENCES Country(CountryID),
  ->   CONSTRAINT fk_employee_job
  ->     FOREIGN KEY (JobID)
  ->     REFERENCES Job(JobID)
  -> );
Query OK, 0 rows affected (0.07 sec)
```

```

mysql> -- =====
mysql> -- INSERT MANAGERS
mysql> -- =====
mysql>
mysql> INSERT INTO Manager VALUES
  -> (501, 'John Smith'),
  -> (502, 'Maria Lopez'),
  -> (503, 'David Lee');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT COUNTRIES
mysql> -- =====
mysql>
mysql> INSERT INTO Country VALUES
  -> (1, 'India'),
  -> (2, 'USA'),
  -> (3, 'Singapore');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT JOBS
mysql> -- =====
mysql>
mysql> INSERT INTO Job VALUES
  -> (101, 'Software Engineer'),
  -> (102, 'HR Manager'),
  -> (103, 'Accountant');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT DEPARTMENTS
mysql> -- =====
mysql>
mysql> INSERT INTO Department VALUES
  -> (10, 'IT Department', 501),
  -> (20, 'Human Resources', 502),
  -> (30, 'Finance', 503);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT EMPLOYEES
mysql> -- =====
mysql>
mysql> INSERT INTO Employee VALUES
  -> (1001, 'Ravi Kumar', 10, 501, 1, 101, 65000.00),
  -> (1002, 'Priya Sharma', 20, 502, 1, 102, 72000.00),
  -> (1003, 'John Brown', 30, 503, 2, 103, 68000.00),
  -> (1004, 'Anita Rao', 10, 501, 3, 101, 70000.00);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- DISPLAY FINAL 3NF TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | ManagerID | CountryID | JobID | Salary |
+-----+-----+-----+-----+-----+-----+
| 1001 | Ravi Kumar | 10 | 501 | 1 | 101 | 65000.00 |
| 1002 | Priya Sharma | 20 | 502 | 1 | 102 | 72000.00 |
| 1003 | John Brown | 30 | 503 | 2 | 103 | 68000.00 |
| 1004 | Anita Rao | 10 | 501 | 3 | 101 | 70000.00 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Department;
+-----+-----+-----+
| DeptID | DeptName | ManagerID |
+-----+-----+-----+
| 10 | IT Department | 501 |
| 20 | Human Resources | 502 |
| 30 | Finance | 503 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql>
mysql> SELECT * FROM Manager;
+-----+-----+
| ManagerID | ManagerName |
+-----+-----+
| 501 | John Smith |
| 502 | Maria Lopez |
| 503 | David Lee |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Country;
+-----+-----+
| CountryID | CountryName |
+-----+-----+
| 1 | India |
| 2 | USA |
| 3 | Singapore |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Job;
+-----+-----+
| JobID | JobTitle |
+-----+-----+
| 101 | Software Engineer |
| 102 | HR Manager |
| 103 | Accountant |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE EMPLOYEE INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
-> e.EmpID,
-> e.EmpName,
-> d.DeptName,
-> m.ManagerName,
-> c.CountryName,
-> j.JobTitle,
-> e.Salary
-> FROM Employee e
-> JOIN Department d
-> ON e.DeptID = d.DeptID
-> JOIN Manager m
-> ON e.ManagerID = m.ManagerID
-> JOIN Country c
-> ON e.CountryID = c.CountryID
-> JOIN Job j
-> ON e.JobID = j.JobID
-> ORDER BY e.EmpID;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptName | ManagerName | CountryName | JobTitle | Salary |
+-----+-----+-----+-----+-----+-----+
| 1001 | Ravi Kumar | IT Department | John Smith | India | Software Engineer | 65000.00 |
| 1002 | Priya Sharma | Human Resources | Maria Lopez | India | HR Manager | 72000.00 |
| 1003 | John Brown | Finance | David Lee | USA | Accountant | 68000.00 |
| 1004 | Anita Rao | IT Department | John Smith | Singapore | Software Engineer | 70000.00 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |

```

6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Given relation

LIBRARY(
MemberID,
MemberName,
BookID,
Title,
Author,
Genre,
BorrowDate
)

1. Case Analysis

Assume:

- MemberID identifies a library member.
- MemberName is determined by MemberID.
- BookID identifies a book.
- BookID determines its title, author, and genre.
- A member can borrow multiple books.
- The borrowing date belongs to a particular member-book borrowing transaction.

Example:

MemberID	MemberName	BookID	Title	Author	Genre	BorrowDate
M01	Ravi	B01	DBMS	Kumar	Technical	2026-08-01
M01	Ravi	B02	Java	James	Programming	2026-08-03
M02	Priya	B01	DBMS	Kumar	Technical	2026-08-05

2. Functional Dependencies

The main FDs are:

MemberID → MemberName

BookID → Title, Author, Genre

$(\text{MemberID}, \text{BookID}) \rightarrow \text{BorrowDate}$

The candidate key is:

$(\text{MemberID}, \text{BookID})$

3. Multivalued Dependency

The important point for 4NF is the independent multi-valued relationship.

A member can have multiple books:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

This means that for a given member, there can be a set of multiple books.

Important assumption

For the 4NF analysis, we assume that a member's collection of borrowed books is an independent multivalued fact from the member's other independent information.

Under this assumption:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

is a multivalued dependency.

4. Why This Causes a 4NF Problem

4NF requires:

For every non-trivial MVD $X \twoheadrightarrow Y$, X must be a superkey.

Here:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

But MemberID alone is not the key of the original relation.

Therefore, the original relation violates 4NF.

5. Decomposition

Separate member information:

$\text{MEMBER}(\text{MemberID}, \text{MemberName})$

Separate book information:

$\text{BOOK}(\text{BookID}, \text{Title}, \text{Author}, \text{Genre})$

Keep the borrowing relationship:

$\text{BORROWING}(\text{MemberID}, \text{BookID}, \text{BorrowDate})$

Final 4NF Schema

MEMBER

MEMBER(MemberID, MemberName)

- MemberID – PK

BOOK

BOOK(BookID, Title, Author, Genre)

- BookID – PK

BORROWING

BORROWING(MemberID, BookID, BorrowDate)

- Composite PK: (MemberID, BookID)
- MemberID – FK → MEMBER(MemberID)
- BookID – FK → BOOK(BookID)

6. Why the Decomposition is in 4NF

The original LIBRARY relation combines member information, book information, and borrowing information in a single table. This results in repeated member and book details and creates redundancy.

After decomposition, the information is separated into:

MEMBER

↓

BORROWING

↑

BOOK

- MEMBER stores member details only.
- BOOK stores book details only.
- BORROWING represents the relationship between members and books, along with the borrowing date.

The multivalued dependency $\text{MemberID} \twoheadrightarrow \text{BookID}$ is separated through the BORROWING relation. Therefore, member and book information is stored only once, reducing redundancy and eliminating the 4NF violation.

Hence, the decomposed relations MEMBER, BOOK, and BORROWING satisfy 4NF.

7. MySQL Implementation

```
mysql> =====
mysql> -- STEP 1: MEMBER TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Member (
  ->     MemberID VARCHAR(10) PRIMARY KEY,
  ->     MemberName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> =====
mysql> -- STEP 2: BOOK TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Book (
  ->     BookID VARCHAR(10) PRIMARY KEY,
  ->     Title VARCHAR(150) NOT NULL,
  ->     Author VARCHAR(100) NOT NULL,
  ->     Genre VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> =====
mysql> -- STEP 3: BORROWING TABLE
mysql> =====
mysql>
mysql> CREATE TABLE Borrowing (
  ->     MemberID VARCHAR(10),
  ->     BookID VARCHAR(10),
  ->     BorrowDate DATE NOT NULL,
  ->     PRIMARY KEY (MemberID, BookID),
  ->     CONSTRAINT fk_borrowing_member
  ->     FOREIGN KEY (MemberID)
  ->     REFERENCES Member(MemberID),
  ->     CONSTRAINT fk_borrowing_book
  ->     FOREIGN KEY (BookID)
  ->     REFERENCES Book(BookID)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> =====
mysql> -- INSERT MEMBERS
mysql> =====
mysql>
mysql> INSERT INTO Member VALUES
  -> ('M01', 'Ravi'),
  -> ('M02', 'Priya'),
  -> ('M03', 'Arun'),
  -> ('M04', 'Kiran');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> =====
mysql> -- INSERT BOOKS
mysql> =====
mysql>
mysql> INSERT INTO Book VALUES
  -> ('B01', 'Database Systems', 'Kumar', 'Technical'),
  -> ('B02', 'Java Programming', 'James', 'Programming'),
  -> ('B03', 'Operating Systems', 'William', 'Technical'),
  -> ('B04', 'Computer Networks', 'Andrew', 'Networking');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> -- =====
mysql> -- INSERT BORROWING DATA
mysql> -- =====
mysql>
mysql> INSERT INTO Borrowing VALUES
-> ('M01', 'B01', '2026-08-01'),
-> ('M01', 'B02', '2026-08-03'),
-> ('M02', 'B01', '2026-08-05'),
-> ('M03', 'B03', '2026-08-06'),
-> ('M04', 'B04', '2026-08-07');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- DISPLAY FINAL 4NF TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Member;
+-----+-----+
| MemberID | MemberName |
+-----+-----+
| M01      | Ravi       |
| M02      | Priya      |
| M03      | Arun       |
| M04      | Kiran      |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Book;
+-----+-----+-----+-----+
| BookID | Title           | Author | Genre |
+-----+-----+-----+-----+
| B01    | Database Systems | Kumar  | Technical |
| B02    | Java Programming | James  | Programming |
| B03    | Operating Systems | William | Technical |
| B04    | Computer Networks | Andrew | Networking |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Borrowing;
+-----+-----+-----+
| MemberID | BookID | BorrowDate |
+-----+-----+-----+
| M01      | B01    | 2026-08-01 |
| M01      | B02    | 2026-08-03 |
| M02      | B01    | 2026-08-05 |
| M03      | B03    | 2026-08-06 |
| M04      | B04    | 2026-08-07 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE BORROWING INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
->     m.MemberID,
->     m.MemberName,
->     b.BookID,
->     b.Title,
->     b.Author,
->     b.Genre,
->     br.BorrowDate
-> FROM Borrowing br
-> JOIN Member m
->     ON br.MemberID = m.MemberID
-> JOIN Book b
->     ON br.BookID = b.BookID
-> ORDER BY m.MemberID, br.BorrowDate;
```

MemberID	MemberName	BookID	Title	Author	Genre	BorrowDate
M01	Ravi	B01	Database Systems	Kumar	Technical	2026-08-01
M01	Ravi	B02	Java Programming	James	Programming	2026-08-03
M02	Priya	B01	Database Systems	Kumar	Technical	2026-08-05
M03	Arun	B03	Operating Systems	William	Technical	2026-08-06
M04	Kiran	B04	Computer Networks	Andrew	Networking	2026-08-07

```
5 rows in set (0.01 sec)

mysql> |
```

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Given Relation

Since the question does not provide attributes, assume the telecom billing schema is:

BILLING(BillID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, CallID, CallDuration, CallCharge)

1. Relation Analysis

The relation stores customer, mobile plan, billing, and call information in one table.

- BillID identifies a bill.
- CustomerID identifies a customer.
- PhoneNo identifies the customer's phone number.
- PlanID identifies the subscribed plan.
- CallID identifies a particular call.
- CallDuration stores call duration.
- CallCharge stores the charge for the call.

2. Candidate Key

Assuming a bill contains multiple calls:

(BillID, CallID)

is the candidate key.

3. Functional Dependencies

BillID $\rightarrow$ CustomerID, PlanID

CustomerID $\rightarrow$ CustomerName, PhoneNo

PlanID $\rightarrow$ PlanName

CallID $\rightarrow$ CallDuration, CallCharge

Therefore:

(BillID, CallID) $\rightarrow$ all attributes

4. Types of Functional Dependencies

1. Full Functional Dependency

$(\text{BillID}, \text{CallID}) \rightarrow \text{CallDuration}$

$(\text{BillID}, \text{CallID}) \rightarrow \text{CallCharge}$

The call information is associated with the complete billing key.

2. Partial Functional Dependency

$\text{BillID} \rightarrow \text{CustomerID}$

$\text{BillID} \rightarrow \text{PlanID}$

$\text{CallID} \rightarrow \text{CallDuration}$

$\text{CallID} \rightarrow \text{CallCharge}$

These depend on only part of the composite key.

3. Transitive Functional Dependency

$\text{BillID} \rightarrow \text{CustomerID}$

$\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}$

Therefore:

$\text{BillID} \rightarrow \text{CustomerName}, \text{PhoneNo}$

Also:

$\text{BillID} \rightarrow \text{PlanID}$

$\text{PlanID} \rightarrow \text{PlanName}$

Therefore:

$\text{BillID} \rightarrow \text{PlanName}$

4. Trivial Functional Dependency

$(\text{BillID}, \text{CallID}) \rightarrow \text{BillID}$

$(\text{BillID}, \text{CallID}) \rightarrow \text{CallID}$

5. Non-Trivial Functional Dependency

$\text{CustomerID} \rightarrow \text{CustomerName}$

$\text{PlanID} \rightarrow \text{PlanName}$

5. Primary Key

Primary Key = $(\text{BillID}, \text{CallID})$

6. Candidate Keys

Candidate Key = $(\text{BillID}, \text{CallID})$

Under the stated assumptions, this is the only candidate key.


```
mysql> -- =====
mysql> -- FIND DISTINCT CUSTOMERS
mysql> -- =====
mysql>
mysql> SELECT DISTINCT
-> CustomerID,
-> CustomerName,
-> PhoneNo
-> FROM Billing;
```

CustomerID	CustomerName	PhoneNo
501	Ravi Kumar	9876543210
502	Priya Sharma	9876501234
503	Arun Kumar	9876512345

3 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- FIND DISTINCT PLANS
mysql> -- =====
mysql>
mysql> SELECT DISTINCT
-> PlanID,
-> PlanName
-> FROM Billing;
```

PlanID	PlanName
101	Unlimited Plan
102	Premium Plan
103	Basic Plan

3 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- DISPLAY BILL DETAILS
mysql> -- =====
mysql>
mysql> SELECT
-> BillID,
-> CustomerID,
-> CustomerName,
-> PhoneNo,
-> PlanID,
-> PlanName,
-> CallID,
-> CallDuration,
-> CallCharge
-> FROM Billing
-> ORDER BY BillID, CallID;
```

BillID	CustomerID	CustomerName	PhoneNo	PlanID	PlanName	CallID	CallDuration	CallCharge
1001	501	Ravi Kumar	9876543210	101	Unlimited Plan	9001	10	5.00
1001	501	Ravi Kumar	9876543210	101	Unlimited Plan	9002	15	7.50
1002	502	Priya Sharma	9876501234	102	Premium Plan	9003	20	10.00
1002	502	Priya Sharma	9876501234	102	Premium Plan	9004	5	2.50
1003	503	Arun Kumar	9876512345	103	Basic Plan	9005	8	4.00

5 rows in set (0.00 sec)

```

mysql>
mysql> -- =====
mysql> -- VERIFY TOTAL CALL CHARGES PER BILL
mysql> -- =====
mysql>
mysql> SELECT
    ->     BillID,
    ->     SUM(CallCharge) AS TotalCallCharge
    -> FROM Billing
    -> GROUP BY BillID;
+-----+-----+
| BillID | TotalCallCharge |
+-----+-----+
|    1001 |             12.50 |
|    1002 |             12.50 |
|    1003 |              4.00 |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- VERIFY NUMBER OF CALLS PER BILL
mysql> -- =====
mysql>
mysql> SELECT
    ->     BillID,
    ->     COUNT(CallID) AS NumberOfCalls
    -> FROM Billing
    -> GROUP BY BillID;
+-----+-----+
| BillID | NumberOfCalls |
+-----+-----+
|    1001 |              2 |
|    1002 |              2 |
|    1003 |              1 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> |

```

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Given Relation

PRODUCT(PID, Pname, SupplierID, SupplierName, Category, Price, Warehouse)

1. Functional Dependencies

$PID \rightarrow Pname, SupplierID, Category, Price, Warehouse$

$SupplierID \rightarrow SupplierName$

2. Types of Functional Dependencies

1. Full Functional Dependency

$PID \rightarrow Pname$

$PID \rightarrow SupplierID$

$PID \rightarrow Category$

$PID \rightarrow Price$

$PID \rightarrow Warehouse$

Since PID is a single-attribute key, these are full dependencies.

2. Partial Functional Dependency

None, because the candidate key is not composite.

3. Transitive Functional Dependency

$PID \rightarrow SupplierID$

$SupplierID \rightarrow SupplierName$

Therefore:

$PID \rightarrow SupplierName$

This is a **Transitive Dependency**.

4. Trivial Functional Dependency

No important trivial FD is required for the normalization of this relation.

5. Non-Trivial Functional Dependency

$PID \rightarrow PName$

$SupplierID \rightarrow SupplierName$

3. Candidate Key

PID

4. Normalization

1NF

All attributes contain atomic values, so the relation satisfies 1NF.

2NF

There is no composite candidate key, so there are no partial dependencies.

Therefore, the relation satisfies 2NF.

3NF

The transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

violates 3NF.

Final 3NF / BCNF Design

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

- $PID \rightarrow PK$
- $SupplierID \rightarrow FK$

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

- $SupplierID \rightarrow PK$

Final Relationship

SUPPLIER 1 —————< PRODUCT

Why it is BCNF

In PRODUCT:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

PID is the key.

In SUPPLIER:

$SupplierID \rightarrow SupplierName$

SupplierID is the key.

Therefore, the final relations satisfy BCNF, which is stronger than 3NF.

5. MySQL Implementation

```
Database changed
mysql>
mysql> -- =====
mysql> -- STEP 1: SUPPLIER TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Supplier (
  ->     SupplierID INT PRIMARY KEY,
  ->     SupplierName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- =====
mysql> -- STEP 2: PRODUCT TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Product (
  ->     PID INT PRIMARY KEY,
  ->     PName VARCHAR(100) NOT NULL,
  ->     SupplierID INT NOT NULL,
  ->     Category VARCHAR(100) NOT NULL,
  ->     Price DECIMAL(10,2) NOT NULL,
  ->     Warehouse VARCHAR(100) NOT NULL,
  ->
  ->     CONSTRAINT fk_product_supplier
  ->     FOREIGN KEY (SupplierID)
  ->     REFERENCES Supplier(SupplierID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> -- =====
mysql> -- INSERT SUPPLIERS
mysql> -- =====
mysql>
mysql> INSERT INTO Supplier VALUES
  -> (101, 'ABC Electronics'),
  -> (102, 'Global Traders'),
  -> (103, 'Tech Suppliers');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT PRODUCTS
mysql> -- =====
mysql>
mysql> INSERT INTO Product VALUES
  -> (1001, 'Laptop', 101, 'Electronics', 65000.00, 'Warehouse A'),
  -> (1002, 'Keyboard', 101, 'Accessories', 1500.00, 'Warehouse A'),
  -> (1003, 'Office Chair', 102, 'Furniture', 8500.00, 'Warehouse B'),
  -> (1004, 'Monitor', 103, 'Electronics', 18000.00, 'Warehouse C');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```



```
mysql> -- =====
mysql> -- DISPLAY SUPPLIER TABLE
mysql> -- =====
mysql>
```

```
mysql> SELECT * FROM Supplier;
```

SupplierID	SupplierName
101	ABC Electronics
102	Global Traders
103	Tech Suppliers

3 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- DISPLAY PRODUCT TABLE
mysql> -- =====
mysql>
mysql> SELECT * FROM Product;
```

PID	PName	SupplierID	Category	Price	Warehouse
1001	Laptop	101	Electronics	65000.00	Warehouse A
1002	Keyboard	101	Accessories	1500.00	Warehouse A
1003	Office Chair	102	Furniture	8500.00	Warehouse B
1004	Monitor	103	Electronics	18000.00	Warehouse C

4 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE PRODUCT INFORMATION
mysql> -- =====
mysql>
```

```
mysql> SELECT
->   p.PID,
->   p.PName,
->   s.SupplierID,
->   s.SupplierName,
->   p.Category,
->   p.Price,
->   p.Warehouse
-> FROM Product p
-> JOIN Supplier s
->   ON p.SupplierID = s.SupplierID
-> ORDER BY p.PID;
```

PID	PName	SupplierID	SupplierName	Category	Price	Warehouse
1001	Laptop	101	ABC Electronics	Electronics	65000.00	Warehouse A
1002	Keyboard	101	ABC Electronics	Accessories	1500.00	Warehouse A
1003	Office Chair	102	Global Traders	Furniture	8500.00	Warehouse B
1004	Monitor	103	Tech Suppliers	Electronics	18000.00	Warehouse C

4 rows in set (0.00 sec)

```
mysql>
```

```
mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE PRODUCT INFORMATION
mysql> -- =====
mysql>
```

```
mysql> SELECT
->   p.PID,
->   p.PName,
->   s.SupplierID,
->   s.SupplierName,
->   p.Category,
->   p.Price,
->   p.Warehouse
-> FROM Product p
-> JOIN Supplier s
->   ON p.SupplierID = s.SupplierID
-> ORDER BY p.PID;
```

PID	PName	SupplierID	SupplierName	Category	Price	Warehouse
1001	Laptop	101	ABC Electronics	Electronics	65000.00	Warehouse A
1002	Keyboard	101	ABC Electronics	Accessories	1500.00	Warehouse A
1003	Office Chair	102	Global Traders	Furniture	8500.00	Warehouse B
1004	Monitor	103	Tech Suppliers	Electronics	18000.00	Warehouse C

4 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- COUNT PRODUCTS FROM EACH SUPPLIER
mysql> -- =====
mysql>
```

```
mysql> SELECT
->   s.SupplierID,
->   s.SupplierName,
->   COUNT(p.PID) AS NumberOfProducts
-> FROM Supplier s
-> LEFT JOIN Product p
->   ON s.SupplierID = p.SupplierID
-> GROUP BY s.SupplierID, s.SupplierName;
```

SupplierID	SupplierName	NumberOfProducts
101	ABC Electronics	2
102	Global Traders	1
103	Tech Suppliers	1

3 rows in set (0.00 sec)

```
mysql> |
```

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Since no specific schema is given, assume the poorly normalized School ERP relation is:

SCHOOL(
StudentID,
StudentName,
ClassID,
ClassName,
TeacherID,
TeacherName,
SubjectID,
SubjectName,
Grade
)

1. Functional Dependencies

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$

$\text{TeacherID} \rightarrow \text{TeacherName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{Grade}$

2. Types of Functional Dependencies

1. Full Functional Dependency

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{Grade}$

Grade depends on the complete composite key.

2. Partial Functional Dependency

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

These attributes depend on only part of the composite key.

3. Transitive Functional Dependency

$\text{ClassID} \rightarrow \text{TeacherID}$

$\text{TeacherID} \rightarrow \text{TeacherName}$

Therefore:

$\text{ClassID} \rightarrow \text{TeacherName}$

This is a transitive dependency.

4. Trivial Functional Dependency

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{StudentID}$

$(\text{StudentID}, \text{SubjectID}) \rightarrow \text{SubjectID}$

5. Non-Trivial Functional Dependency

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

$\text{TeacherID} \rightarrow \text{TeacherName}$

3. Candidate Key

Assuming a student receives one grade for each subject:

$(\text{StudentID}, \text{SubjectID})$

4. Normalization

1NF

All attributes contain atomic values.

2NF

Partial dependencies exist:

$\text{StudentID} \rightarrow \text{StudentName}$

$\text{SubjectID} \rightarrow \text{SubjectName}$

Therefore, decomposition is required.

3NF

The transitive dependency:

$\text{ClassID} \rightarrow \text{TeacherID} \rightarrow \text{TeacherName}$

must be removed.

5. BCNF Decomposition

STUDENT

STUDENT(StudentID, StudentName)

CLASS

CLASS(ClassID, ClassName, TeacherID)

TEACHER

TEACHER(TeacherID, TeacherName)

SUBJECT

SUBJECT(SubjectID, SubjectName)

ENROLLMENT

ENROLLMENT(StudentID, SubjectID, Grade)

6. Keys

Relation	Primary Key
STUDENT	StudentID
CLASS	ClassID
TEACHER	TeacherID
SUBJECT	SubjectID
ENROLLMENT	(StudentID, SubjectID)

7. Relationships

STUDENT 1 ———< ENROLLMENT >———— 1 SUBJECT

TEACHER 1 ———< CLASS

8. BCNF Justification

Each determinant in the decomposed relations is a candidate key:

StudentID → StudentName

ClassID → ClassName, TeacherID

TeacherID → TeacherName

SubjectID → SubjectName

(StudentID, SubjectID) → Grade

Therefore, the resulting relations satisfy BCNF.

9. MySQL Implementation

```
MySQL 8.0 Command Line Cli  ×  +  ▾

mysql> -- =====
mysql> -- STEP 1: STUDENT TABLE
mysql> -- =====
mysql> CREATE TABLE Student (
  → StudentID INT PRIMARY KEY,
  → StudentName VARCHAR(100) NOT NULL
  → );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- =====
mysql> -- STEP 2: TEACHER TABLE
mysql> -- =====
mysql> CREATE TABLE Teacher (
  → TeacherID INT PRIMARY KEY,
  → TeacherName VARCHAR(100) NOT NULL
  → );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- =====
mysql> -- STEP 3: CLASS TABLE
mysql> -- =====
mysql> CREATE TABLE Class (
  → ClassID INT PRIMARY KEY,
  → ClassName VARCHAR(100) NOT NULL,
  → TeacherID INT NOT NULL,
  →
  → CONSTRAINT fk_class_teacher
  → FOREIGN KEY (TeacherID)
  → REFERENCES Teacher(TeacherID)
  → );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> -- =====
mysql> -- STEP 4: SUBJECT TABLE
mysql> -- =====
mysql> CREATE TABLE Subject (
  → SubjectID INT PRIMARY KEY,
  → SubjectName VARCHAR(100) NOT NULL
  → );
Query OK, 0 rows affected (0.03 sec)
```

```
MySQL 8.0 Command Line Cli  ×  +  ▾

mysql> -- =====
mysql> -- STEP 5: ENROLLMENT TABLE
mysql> -- =====
mysql> CREATE TABLE Enrollment (
  → StudentID INT,
  → SubjectID INT,
  → Grade VARCHAR(5),
  →
  → PRIMARY KEY (StudentID, SubjectID),
  →
  → CONSTRAINT fk_enrollment_student
  → FOREIGN KEY (StudentID)
  → REFERENCES Student(StudentID),
  →
  → CONSTRAINT fk_enrollment_subject
  → FOREIGN KEY (SubjectID)
  → REFERENCES Subject(SubjectID)
  → );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- =====
mysql> -- INSERT STUDENTS
mysql> -- =====
mysql> INSERT INTO Student VALUES
  → (1001, 'Ravi'),
  → (1002, 'Priya'),
  → (1003, 'Arun'),
  → (1004, 'Kiran');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT TEACHERS
mysql> -- =====
mysql> INSERT INTO Teacher VALUES
  → (501, 'Mr. Kumar'),
  → (502, 'Ms. Meena'),
  → (503, 'Mr. Raj');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT CLASSES
mysql> -- =====
mysql> INSERT INTO Class VALUES
  → (10, 'Class 10-A', 501),
  → (20, 'Class 10-B', 502),
  → (30, 'Class 10-C', 503);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```

mysql> -- =====
mysql> -- INSERT SUBJECTS
mysql> -- =====
mysql>
mysql> INSERT INTO Subject VALUES
    -> (101, 'Mathematics'),
    -> (102, 'Science'),
    -> (103, 'Computer Science'),
    -> (104, 'English');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT ENROLLMENTS
mysql> -- =====
mysql>
mysql> INSERT INTO Enrollment VALUES
    -> (1001, 101, 'A'),
    -> (1001, 102, 'B+'),
    -> (1002, 101, 'A+'),
    -> (1002, 103, 'A'),
    -> (1003, 104, 'B'),
    -> (1004, 103, 'A+');
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- DISPLAY ALL FINAL TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Student;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|      1001 | Ravi        |
|      1002 | Priya       |
|      1003 | Arun        |
|      1004 | Kiran       |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Teacher;
+-----+-----+
| TeacherID | TeacherName |
+-----+-----+
|      501 | Mr. Kumar   |
|      502 | Ms. Meena   |
|      503 | Mr. Raj     |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Class;
+-----+-----+-----+
| ClassID | ClassName | TeacherID |
+-----+-----+-----+
|      10 | Class 10-A |      501   |
|      20 | Class 10-B |      502   |
|      30 | Class 10-C |      503   |
+-----+-----+-----+
3 rows in set (0.00 sec)

```



```

mysql>
mysql> -- =====
mysql> -- DISPLAY COMPLETE STUDENT INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
->     s.StudentID,
->     s.StudentName,
->     sub.SubjectID,
->     sub.SubjectName,
->     e.Grade
-> FROM Enrollment e
-> JOIN Student s
->     ON e.StudentID = s.StudentID
-> JOIN Subject sub
->     ON e.SubjectID = sub.SubjectID
-> ORDER BY s.StudentID, sub.SubjectID;

```

StudentID	StudentName	SubjectID	SubjectName	Grade
1001	Ravi	101	Mathematics	A
1001	Ravi	102	Science	B+
1002	Priya	101	Mathematics	A+
1002	Priya	103	Computer Science	A
1003	Arun	104	English	B
1004	Kiran	103	Computer Science	A+

6 rows in set (0.00 sec)

```

mysql>
mysql> -- =====
mysql> -- DISPLAY CLASS AND TEACHER INFORMATION
mysql> -- =====
mysql>
mysql> SELECT
->     c.ClassID,
->     c.ClassName,
->     t.TeacherID,
->     t.TeacherName
-> FROM Class c
-> JOIN Teacher t
->     ON c.TeacherID = t.TeacherID
-> ORDER BY c.ClassID;

```

ClassID	ClassName	TeacherID	TeacherName
10	Class 10-A	501	Mr. Kumar
20	Class 10-B	502	Ms. Meena
30	Class 10-C	503	Mr. Raj

3 rows in set (0.00 sec)

```

mysql>

```

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Assumed Relation

Because the question does not provide the attributes, consider:

```
STREAMING(  
  MovieID,  
  MovieTitle,  
  ActorID,  
  ActorName,  
  PlatformID,  
  PlatformName  
)
```

Assume:

- A movie can have multiple actors.
- A movie can be available on multiple platforms.
- An actor can appear in movies available on multiple platforms.
- The three relationships are independent.

1. Functional Dependencies

$\text{MovieID} \rightarrow \text{MovieTitle}$

$\text{ActorID} \rightarrow \text{ActorName}$

$\text{PlatformID} \rightarrow \text{PlatformName}$

2. Candidate Key

Under the stated independent many-to-many-to-many assumption:

(MovieID, ActorID, PlatformID)

Types of Functional Dependencies

1. Full Functional Dependency

(MovieID, ActorID, PlatformID) $\rightarrow$ relationship occurrence

The complete combination identifies a movie-actor-platform association.

2. Partial Functional Dependency

$\text{MovieID} \rightarrow \text{MovieTitle}$

$\text{ActorID} \rightarrow \text{ActorName}$

$\text{PlatformID} \rightarrow \text{PlatformName}$

Each descriptive attribute depends on only part of the composite key.

3. Transitive Dependency

No important transitive dependency is explicitly required in this simplified case.

4. Trivial Functional Dependency

$(\text{MovieID}, \text{ActorID}, \text{PlatformID}) \rightarrow \text{MovieID}$

$(\text{MovieID}, \text{ActorID}, \text{PlatformID}) \rightarrow \text{ActorID}$

$(\text{MovieID}, \text{ActorID}, \text{PlatformID}) \rightarrow \text{PlatformID}$

3. Multivalued Dependencies

The independent relationships can be represented as:

$\text{MovieID} \twoheadrightarrow \text{ActorID}$

$\text{MovieID} \twoheadrightarrow \text{PlatformID}$

These indicate that a movie can have multiple actors and multiple platforms independently.

However, the main issue for 5NF is the join dependency.

4. Join Dependency

The original relation can be reconstructed from:

$\text{MOVIE_ACTOR}(\text{MovieID}, \text{ActorID})$

$\text{MOVIE_PLATFORM}(\text{MovieID}, \text{PlatformID})$

$\text{ACTOR_PLATFORM}(\text{ActorID}, \text{PlatformID})$

Conceptually:

STREAMING

=

MOVIE_ACTOR

$\bowtie$

MOVIE_PLATFORM

$\bowtie$

ACTOR_PLATFORM

This is a join dependency.

5. 5NF Decomposition

MOVIE

MOVIE(MovieID, MovieTitle)

- MovieID $\rightarrow$ PK

ACTOR

ACTOR(ActorID, ActorName)

- ActorID $\rightarrow$ PK

PLATFORM

PLATFORM(PlatformID, PlatformName)

- PlatformID $\rightarrow$ PK

MOVIE_ACTOR

MOVIE_ACTOR(MovieID, ActorID)

- Composite PK: (MovieID, ActorID)

MOVIE_PLATFORM

MOVIE_PLATFORM(MovieID, PlatformID)

- Composite PK: (MovieID, PlatformID)

ACTOR_PLATFORM

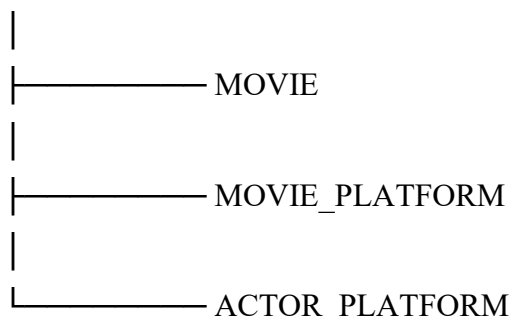
ACTOR_PLATFORM(ActorID, PlatformID)

- Composite PK: (ActorID, PlatformID)

6. Lossless-Join Verification

The decomposition is considered lossless if joining the decomposed relations reconstructs exactly the valid tuples of the original relation without losing information.

MOVIE_ACTOR



The join:

MOVIE_ACTOR

⋈ MOVIE_PLATFORM

⋈ ACTOR_PLATFORM

reconstructs the original STREAMING relationship under the stated join-dependency assumption.

Conclusion

The original movie-streaming relation contains independent relationships among movies, actors,

and platforms. These relationships create a join dependency. Decomposing the relation into smaller relations eliminates the redundancy and produces a 5NF design where further non-lossy decomposition based on the stated join dependency is not required.

7. MySQL Implementation

```
MySQL 8.0 Command Line Cli  ×  +  ∨

mysql> -- =====
mysql> -- STEP 1: MOVIE TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Movie (
  ->     MovieID INT PRIMARY KEY,
  ->     MovieTitle VARCHAR(150) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- =====
mysql> -- STEP 2: ACTOR TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Actor (
  ->     ActorID INT PRIMARY KEY,
  ->     ActorName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- =====
mysql> -- STEP 3: PLATFORM TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Platform (
  ->     PlatformID INT PRIMARY KEY,
  ->     PlatformName VARCHAR(100) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- =====
mysql> -- STEP 4: MOVIE_ACTOR TABLE
mysql> -- =====
mysql>
mysql> CREATE TABLE Movie_Actor (
  ->     MovieID INT,
  ->     ActorID INT,
  ->
  ->     PRIMARY KEY (MovieID, ActorID),
  ->
  ->     CONSTRAINT fk_movie_actor_movie
  ->         FOREIGN KEY (MovieID)
  ->         REFERENCES Movie(MovieID),
  ->
  ->     CONSTRAINT fk_movie_actor_actor
  ->         FOREIGN KEY (ActorID)
  ->         REFERENCES Actor(ActorID)
  -> );
Query OK, 0 rows affected (0.05 sec)
```

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> -- =====
mysql> -- STEP 5: MOVIE_PLATFORM TABLE
mysql> -- =====
mysql> CREATE TABLE Movie_Platform (
  ->   MovieID INT,
  ->   PlatformID INT,
  ->   PRIMARY KEY (MovieID, PlatformID),
  ->   CONSTRAINT fk_movie_platform_movie
  ->     FOREIGN KEY (MovieID)
  ->     REFERENCES Movie(MovieID),
  ->   CONSTRAINT fk_movie_platform_platform
  ->     FOREIGN KEY (PlatformID)
  ->     REFERENCES Platform(PlatformID)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- =====
mysql> -- STEP 6: ACTOR_PLATFORM TABLE
mysql> -- =====
mysql> CREATE TABLE Actor_Platform (
  ->   ActorID INT,
  ->   PlatformID INT,
  ->   PRIMARY KEY (ActorID, PlatformID),
  ->   CONSTRAINT fk_actor_platform_actor
  ->     FOREIGN KEY (ActorID)
  ->     REFERENCES Actor(ActorID),
  ->   CONSTRAINT fk_actor_platform_platform
  ->     FOREIGN KEY (PlatformID)
  ->     REFERENCES Platform(PlatformID)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- =====
mysql> -- INSERT MOVIES
mysql> -- =====
mysql> INSERT INTO Movie VALUES
  -> (1, 'The Adventure'),
  -> (2, 'Future World'),
  -> (3, 'Hidden Truth');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT ACTORS
mysql> -- =====
mysql> INSERT INTO Actor VALUES
  -> (101, 'Arun Kumar'),
  -> (102, 'Priya Rao'),
  -> (103, 'Kiran Das');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
MySQL 8.0 Command Line Cli x + v

mysql> -- =====
mysql> -- INSERT PLATFORMS
mysql> -- =====
mysql>
mysql> INSERT INTO Platform VALUES
  -> (201, 'StreamPlus'),
  -> (202, 'MovieHub'),
  -> (203, 'ViewNow');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT MOVIE-ACTOR RELATIONSHIPS
mysql> -- =====
mysql>
mysql> INSERT INTO Movie_Actor VALUES
  -> (1, 101),
  -> (1, 102),
  -> (2, 102),
  -> (2, 103),
  -> (3, 101);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT MOVIE-PLATFORM RELATIONSHIPS
mysql> -- =====
mysql>
mysql> INSERT INTO Movie_Platform VALUES
  -> (1, 201),
  -> (1, 202),
  -> (2, 202),
  -> (2, 203),
  -> (3, 201);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> -- =====
mysql> -- INSERT ACTOR-PLATFORM RELATIONSHIPS
mysql> -- =====
mysql>
mysql> INSERT INTO Actor_Platform VALUES
  -> (101, 201),
  -> (101, 202),
  -> (102, 201),
  -> (102, 202),
  -> (102, 203),
  -> (103, 202),
  -> (103, 203);
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli x + v

mysql> -- =====
mysql> -- DISPLAY FINAL 5NF TABLES
mysql> -- =====
mysql>
mysql> SELECT * FROM Movie;
+-----+-----+
| MovieID | MovieTitle |
+-----+-----+
| 1       | The Adventure |
| 2       | Future World |
| 3       | Hidden Truth |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Actor;
+-----+-----+
| ActorID | ActorName |
+-----+-----+
| 101     | Arun Kumar |
| 102     | Priya Rao |
| 103     | Kiran Das |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Platform;
+-----+-----+
| PlatformID | PlatformName |
+-----+-----+
| 201        | StreamPlus |
| 202        | MovieHub |
| 203        | ViewNow |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Movie_Actor;
+-----+-----+
| MovieID | ActorID |
+-----+-----+
| 1       | 101 |
| 3       | 101 |
| 1       | 102 |
| 2       | 102 |
| 2       | 103 |
+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Movie_Platform;
+-----+-----+
| MovieID | PlatformID |
+-----+-----+
| 1       | 201 |
| 3       | 201 |
| 1       | 202 |
| 2       | 202 |
| 2       | 203 |
+-----+-----+
5 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  ×  +  ▾

mysql> SELECT * FROM Actor_Platform;

+----+-----+
| ActorID | PlatformID |
+----+-----+
| 101     | 201        |
| 102     | 201        |
| 101     | 202        |
| 102     | 202        |
| 103     | 202        |
| 102     | 203        |
| 103     | 203        |
+----+-----+
7 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- DISPLAY MOVIES WITH ACTORS
mysql> -- =====
mysql>
mysql> SELECT
  -> m.MovieID,
  -> m.MovieTitle,
  -> a.ActorID,
  -> a.ActorName
  -> FROM Movie_Actor ma
  -> JOIN Movie m
  -> ON ma.MovieID = m.MovieID
  -> JOIN Actor a
  -> ON ma.ActorID = a.ActorID
  -> ORDER BY m.MovieID, a.ActorID;

+----+-----+-----+-----+
| MovieID | MovieTitle | ActorID | ActorName |
+----+-----+-----+-----+
| 1       | The Adventure | 101     | Arun Kumar |
| 1       | The Adventure | 102     | Priya Rao  |
| 2       | Future World | 102     | Priya Rao  |
| 2       | Future World | 103     | Kiran Das  |
| 3       | Hidden Truth | 101     | Arun Kumar |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- =====
mysql> -- DISPLAY MOVIES AVAILABLE ON PLATFORMS
mysql> -- =====
mysql>
mysql> SELECT
  -> m.MovieID,
  -> m.MovieTitle,
  -> p.PlatformID,
  -> p.PlatformName
  -> FROM Movie_Platform mp
  -> JOIN Movie m
  -> ON mp.MovieID = m.MovieID
  -> JOIN Platform p
  -> ON mp.PlatformID = p.PlatformID
  -> ORDER BY m.MovieID, p.PlatformID;

+----+-----+-----+-----+
| MovieID | MovieTitle | PlatformID | PlatformName |
+----+-----+-----+-----+
| 1       | The Adventure | 201        | StreamPlus  |
| 1       | The Adventure | 202        | MovieHub    |
| 2       | Future World | 202        | MovieHub    |
| 2       | Future World | 203        | ViewNow     |
| 3       | Hidden Truth | 201        | StreamPlus  |
+----+-----+-----+-----+
```


7 rows in set (0.00 sec)

```
mysql>
mysql> -- =====
mysql> -- DISPLAY ACTORS AND THEIR PLATFORMS
mysql> -- =====
mysql>
mysql> SELECT
->     a.ActorID,
->     a.ActorName,
->     p.PlatformID,
->     p.PlatformName
-> FROM Actor_Platform ap
-> JOIN Actor a
->     ON ap.ActorID = a.ActorID
-> JOIN Platform p
->     ON ap.PlatformID = p.PlatformID
-> ORDER BY a.ActorID, p.PlatformID;
+-----+-----+-----+-----+
| ActorID | ActorName | PlatformID | PlatformName |
+-----+-----+-----+-----+
|      101 | Arun Kumar |          201 | StreamPlus   |
|      101 | Arun Kumar |          202 | MovieHub     |
|      102 | Priya Rao  |          201 | StreamPlus   |
|      102 | Priya Rao  |          202 | MovieHub     |
|      102 | Priya Rao  |          203 | ViewNow      |
|      103 | Kiran Das  |          202 | MovieHub     |
|      103 | Kiran Das  |          203 | ViewNow      |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> |
```